

Mundwerk — Projektplan

Aussprache-App für Deutsch-Lernende: interaktive Korrektur der Eigenproduktion per Formantanalyse. Team: Kirsten Jänich (Phonetik/DaF), Thomas Pfuhl (Computerlinguistik).

Architektur

Android (Kotlin)

Server (Django/DRF)



Kernentscheidung: Praat wird nicht als externes Programm angesprochen, sondern über **Parselmouth** (Python-Binding, bettet den Praat-Kern ein) — die gesamte Pipeline läuft in einem Python-Prozess und liest auch die TextGrids des Aligners.

Die drei fachlichen Kernprobleme

1. Segmentierung (Forced Alignment)

Bevor Formanten verglichen werden können, muss bekannt sein, wo im Signal z. B. das /ø:/ von „schön“ liegt.

- **Montreal Forced Aligner (MFA)** — Empfehlung. Läuft lokal auf dem Server, deutsches Akustikmodell + Lexikon vorhanden, IPA-kompatibel, bringt G2P mit (wichtig für das Userlexikon in Phase 5).
- Alternative: WebMAUS (BAS München) — exzellent für Deutsch, aber externe Abhängigkeit und für kommerzielle Nutzung lizenzpflichtig.

2. Sprechernormalisierung

F1/F2 sind zwischen Kind, Frau, Mann absolut nicht vergleichbar (Vokaltraktlänge). Lösung: kurze **Kalibrierungsphase beim Onboarding** (User spricht /a: i: u:/), daraus Lobanov-Normalisierung oder Skalierungsfaktor. Ohne Normalisierung: Fehlalarme am Fließband. Außerdem: `maximum_formant` in der Analyse sprecherabhängig setzen (σ ~5000 Hz, φ ~5500 Hz, Kinder höher).

3. Feedback-Mapping (der USP)

Regelbasierte Tabelle: Abweichungsrichtung → artikulatorischer Hinweis.

Befund	Feedback
F1 zu hoch bei /i:/	„Mund zu offen, Zunge höher“
F2 zu niedrig bei /y:/	typischer Fehler /u:/: „Zungenposition wie bei ,ie‘, nur Lippen runden“
F2 zu hoch bei /u:/	„Lippen stärker runden, Zunge weiter hinten“

Das ist das eigentliche Expertenwissen (Kirstens Domäne). Später: L1-spezifische Fehlerhypothesen priorisieren.

Datenmodell (Django, Skizze)

```
class Item(models.Model):
    # Wort oder Satz
    text = models.CharField(max_length=200)
    ipa = models.CharField(max_length=200)
    level = models.CharField(max_length=10) # A1..C2
    focus_segments = models.JSONField() # [{"ø": "ø"}, {"ç": "ç"}]
```

```
class TargetSegment(models.Model): # Expertenwissen
    phone = models.CharField(max_length=10) # IPA
    f1_mean = models.FloatField(); f1_sd = models.FloatField()
    f2_mean = models.FloatField(); f2_sd = models.FloatField()
    feedback_rules = models.JSONField()
```

```
class Recording(models.Model):
    user = models.ForeignKey(User, ...)
    item = models.ForeignKey(Item, ...)
    audio = models.FileField()
    result = models.JSONField(null=True) # Alignment + Formanten + Rating
```

API

```
POST /api/recordings/ (multipart: item_id, audio.wav)
  → 202, task_id (Analyse asynchron: Celery/Redis, MFA braucht 1–3 s)
GET /api/recordings/{id}/ → { rating, segments: [{phone, f1, f2, target,
  deviation, feedback}] }
GET /api/items/?level=A2
```

Analyse-Pipeline (Kern)

```
import parselmouth

snd = parselmouth.Sound("rec.wav")
formants = snd.to_formant_burg(max_number_of_formants=5,
                               maximum_formant=5500) # sprecherabhängig!
# Pro Segment (aus MFA-TextGrid): Median von F1/F2 über das mittlere
# Drittel des Segments (stabiler gegen Transitionen als ein Einzelmesspunkt)
```

Phasenplan

- **Phase 0 (jetzt): Offline-Validierung.** Skript, das WAV → Alignment → Formanten → Vergleich durchspielt. Mit eigenen Aufnahmen (gute vs. absichtlich falsche Aussprache) testen. **Wenn die Diskriminierung hier nicht funktioniert, funktioniert die App nicht.** → server/validation/ in diesem Repo.
- **Phase 1 (MVP):** Nur lange Vokale in Einzelwörtern (/i: y: u: e: ø: o: a:/) — dort ist Formantanalyse am zuverlässigsten. Feste Wortliste, Kalibrierung, Ampel-Rating + ein Hinweissatz. Android: Aufnahme (AudioRecord, 16 kHz mono WAV), Upload (Retrofit), Feedback-Screen. Django-Backend mit obigem Datenmodell.
- **Phase 2:** Kurzvokale, Umlaut-Minimalpaare, Vokalviereck-Visualisierung (Ist vs. Soll als Punkt im F1/F2-Raum).
- **Phase 3:** Konsonanten — Formanten reichen nicht: Frikative brauchen spektrale Momente (CoG für /ç/ vs. /ʃ/), Plosive VOT. Parselmouth kann das, aber es sind andere Messgrößen.
- **Phase 4:** Sätze/Prosodie: Pitch-Kontur (snd.to_pitch()), Vergleich mit Referenzkontur per DTW (Akzent, Satzmelodie).
- **Phase 5:** L1-spezifische Übungspfade, Userlexikon via G2P (MFA).

Offene Punkte

- Referenz-Formantwerte für deutsche Vokale: Startpunkt sind publizierte Mittelwerte (z. B. Pätzold & Simpson 1997, Kiel Corpus); langfristig eigene Referenzdaten von Kirsten/Thomas einpflegen.
- Audio-Format Android → Server: unkomprimiertes WAV 16 kHz/16 bit mono (Formantanalyse verträgt Lossy-Artefakte schlecht — kein AMR/Opus).
- Datenschutz: Aufnahmen sind personenbezogene Daten → Löschkonzept, DSGVO.